GA GUARDIAN

Gamma

Gamma - MultiPositionManager

Security Assessment

March 5th, 2026

Summary

Audit Firm Guardian

Prepared By Michael Lett, CrypticDefense, Jakub Zmyslowski, 0xCiphky

Client Firm Gamma

Final Report Date March 4, 2026

Audit Summary

Gamma engaged Guardian to review the security of their Gamma - MultiPositionManager. From the 2nd of January 2026 to the 4th of February 2026, a team of 4 auditors reviewed the source code in scope. All findings have been recorded in the following report.

Confidence Ranking

Given the minimal High and Critical issues detected during the main review, Guardian assigns a Confidence Ranking of 4 to the protocol. Guardian advises the protocol to consider periodic review with future changes. For detailed understanding of the Guardian Confidence Ranking, please see the rubric on the following page.

Note: Fixes to the findings uncovered in the remediation round 2 review have not been reviewed by Guardian.

✅ Verify the authenticity of this report on Guardian's GitHub: https://github.com/guardianaudits

Guardian Confidence Ranking

Confidence Ranking	Definition and Recommendation	Risk Profile
5: Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. **Recommendation:** Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4: High Confidence	Code is clean, well-structured, and adheres to best practices. Only 1 Significant issue was uncovered per week. Design patterns are sound, and test coverage is strong. **Recommendation:** Suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity issues. Varied Low severity findings.
3: Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas (e.g., weak modularity, risky patterns). No critical design flaws, though some patterns could lead to issues in edge cases. **Recommendation:** Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1-2 High/Critical findings per engagement week.
2: Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities. Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. **Recommendation:** Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week. Or additional High/Critical findings uncovered in remediation review which have not been resolved and confirmed by Guardian.
1: Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. **Recommendation:** Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥5 High/Critical findings and overall systemic flaws.

Table of Contents

Project Information

Project Overview 5

Audit Scope & Methodology 6

Smart Contract Risk Assessment

Findings & Resolutions 9

Addendum

Disclaimer 100

About Guardian 101

Project Overview

Project Summary

Project Name	Gamma
Language	Solidity
Codebase	https://github.com/GammaStrategies/TokenLaunch-OrderbookFactory-MultiPositionManager
Commit(s)	Main Review commit: a75e25f78f0600d500f22fe3d7bd6e74c9955b0e Remediation Review commit: c1890ffcf3655a22c7bb444b87b3aa9fa7bd3a31 Remediation 2 Review commit: f248c7ab5be9e7cafe25d2f7fac0ea2987d997f4

Audit Summary

Delivery Date	March 4, 2026
Audit Methodology	Static Analysis, Manual Review, Test Suite, Contract Fuzzing

Vulnerability Summary

Vulnerability Level	Total	Pending	Declined	Acknowledged	Partially Resolved	Resolved
Critical	0	0	0	0	0	0
High	1	0	0	0	0	1
Medium	22	0	0	0	2	20
Low	17	0	0	4	0	13
Info	29	0	0	17	0	12

Audit Scope & Methodology

```
contract,source,total,comment
src/MultiPositionManager/MultiPositionDeployer.sol,25,71,37
src/MultiPositionManager/MultiPositionFactory.sol,178,368,145
src/MultiPositionManager/MultiPositionManager.sol,285,427,93
src/MultiPositionManager/Relayer.sol,371,777,291
src/MultiPositionManager/RelayerDeployer.sol,42,80,34
src/MultiPositionManager/RelayerFactory.sol,264,531,193
src/LimitOrderBook/OrderBookFactory.sol,500,766,175
src/LaunchPad/SuperchainLBPStrategy.sol,135,211,43
src/LaunchPad/SuperchainLBPStrategyDeployer.sol,43,74,28
src/LaunchPad/SuperchainLBPStrategyFactory.sol,75,135,38
src/LaunchPad/TokenLaunchFactory.sol,123,227,76
src/MultiPositionManager/base/Multicall.sol,21,40,13
src/MultiPositionManager/base/SharedStructs.sol,37,58,27
src/MultiPositionManager/libraries/DepositLogic.sol,214,363,106
src/MultiPositionManager/libraries/PoolManagerUtils.sol,339,434,26
src/MultiPositionManager/libraries/PositionLogic.sol,365,573,142
src/MultiPositionManager/libraries/RebalanceLogic.sol,852,1384,347
src/MultiPositionManager/libraries/RelayerLogic.sol,248,405,110
src/MultiPositionManager/libraries/WithdrawLogic.sol,343,620,194
src/LimitOrderBook/errors/Errors.sol,7,9,1
src/LaunchPad/libraries/LBPDeploymentLib.sol,42,62,16
source count: {
  total: 7615,
  source: 4509,
  comment: 2135,
  single: 972,
  block: 1163,
  mixed: 57,
  empty: 1028,
  todo: 0,
  blockEmpty: 0,
  commentToSourceRatio: 0.4734974495453537
}
```

Audit Scope & Methodology

Vulnerability Classifications

Severity	Impact: *High*	Impact: *Medium*	Impact: *Low*
Likelihood: *High*	● Critical	● High	● Medium
Likelihood: *Medium*	● High	● Medium	● Low
Likelihood: *Low*	● Medium	● Low	● Low

Impact

High Significant loss of assets in the protocol, significant harm to a group of users, or a core functionality of the protocol is disrupted.

Medium A small amount of funds can be lost or ancillary functionality of the protocol is affected. The user or protocol may experience reduced or delayed receipt of intended funds.

Low Can lead to any unexpected behavior with some of the protocol's functionalities that is notable but does not meet the criteria for a higher severity.

Likelihood

High The attack is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount gained or the disruption to the protocol.

Medium An attack vector that is only possible in uncommon cases or requires a large amount of capital to exercise relative to the amount gained or the disruption to the protocol.

Low Unlikely to ever occur in production.

Audit Scope & Methodology

Methodology

Guardian is the ultimate standard for Smart Contract security. An engagement with Guardian entails the following:

- Two competing teams of Guardian security researchers performing an independent review.
- A dedicated fuzzing engineer to construct a comprehensive stateful fuzzing suite for the project.
- An engagement lead security researcher coordinating the 2 teams, performing their own analysis, relaying findings to the client, and orchestrating the testing/verification efforts.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross-referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts. Comprehensive written tests as a part of a code coverage testing suite.
- Contract fuzzing for increased attack resilience.

Findings & Resolutions

ID	Title	Category	Severity	Status
H-01	Missing Deployer Access Control Causes DoS	DoS	High	Resolved
M-01	Unbounded Loop In deployMultiPositionManager	DoS	Medium	Resolved
M-02	RebalanceSwap Skips Protocol Fee Split	Logical Error	Medium	Resolved
M-03	ETH Accounting Breaks In Multicall	Logical Error	Medium	Resolved
M-04	withdrawCustom Reverts On Fee Collection	DoS	Medium	Resolved
M-05	Zero Fee Can Brick Core Functions	Math	Medium	Resolved
M-06	Aggregator Address Not Validated	Validation	Medium	Resolved
M-07	ERC20 Shares Not Redeemable By Holders	Logical Error	Medium	Partially Resolved
M-08	Rebalance Overflow When Price Near Lower Tick	Logical Error	Medium	Resolved
M-09	Single-Token Withdraw DoS With Limit Positions	DoS	Medium	Resolved
M-10	Relayer Owner Desync From MPM Owner	Configuration	Medium	Resolved

Findings & Resolutions

ID	Title	Category	Severity	Status
M-11	Missing Token0 Fallback When liquidityFrom1 Is 0	Logical Error	Medium	Resolved
M-12	Relayer Off-By-One Ratio Extraction	Logical Error	Medium	Resolved
M-13	Unaligned TWAP Center Tick	Logical Error	Medium	Resolved
M-14	Lens Ignores Fee Claim Before Rebalance	Logical Error	Medium	Resolved
M-15	Slippage Bypass When Limit Position 0 Empty	Logical Error	Medium	Resolved
M-16	Rebalance Slippage Ignores Principal Vs Fees	Error	Medium	Resolved
M-17	Gas Refund Ignores L1 Data Fee	Unexpected Behavior	Medium	Partially Resolved
M-18	limitPositionsLength Can Skip Active Limit Slot	Logical Error	Medium	Resolved
M-29	Carpet Blocks Relayer Single-Token Withdraw	Logical Error	Medium	Resolved
M-20	Proportional Mode Handled Inconsistently	Logical Error	Medium	Resolved
M-21	rebalanceSwap Bypasses Carpet Checks	Unexpected Behavior	Medium	Resolved
M-22	Unreachable Single-Token Withdrawal Rebalance	Logical Error	Medium	Resolved

Findings & Resolutions

ID	Title	Category	Severity	Status
L-01	Relayer Deployment Griefing	DoS	Low	Resolved
L-02	Deposit Function Lacks Slippage Protection	Error	Low	Acknowledged
L-03	TWAP Validation Mismatch After Deployment	Configuration	Low	Resolved
L-04	useRebalanceSwap Not Enforced On-Chain	Unexpected Behavior	Low	Resolved
L-05	Incomplete Limit Width Collision Check	Error	Low	Resolved
L-06	TWAP Protection Gas Not Reimbursed	Logical Error	Low	Resolved
L-07	Native Currency Breaks getUniqueTokenPairs()	Logical Error	Low	Resolved
L-08	Proportional Rebalance Can Revert At Min Price	DoS	Low	Resolved
L-09	User-Specified Weights Silently Overridden	Logical Error	Low	Resolved
L-10	ExactOut Swaps Assume Full Input Consumed	Logical Error	Low	Acknowledged
L-11	Dust Pricing Can Brick withdrawCustom	DoS	Low	Resolved
L-12	Relayer Deploy Lacks Ratio Validation	Configuration	Low	Resolved
L-13	Relayer Address Computation Uses Arbitrary Owner	Logical Error	Low	Resolved

Findings & Resolutions

ID	Title	Category	Severity	Status
L-14	Owner Can Block Protocol claimFee For ETH Pools	DoS	Low	Resolved
L-15	Owner Can Blacklist Protocol Fee Recipient	DoS	Low	Acknowledged
L-16	Carpet Mode Reverts On Small Deposits	DoS	Low	Resolved
L-17	MPM Fee Revenue Bypassed Via Custom Hook Pools	Compatibility	Low	Acknowledged
I-01	Unused ETH Causes Revert On Deposits	DoS	Info	Acknowledged
I-02	Missing Deadline Parameter On MPM Functions	Validation	Info	Acknowledged
I-03	Dead Code In scaleAllocations Function	Best Practices	Info	Acknowledged
I-04	Redundant Loop In scaleAllocations	Gas Optimization	Info	Acknowledged
I-05	Unused Slippage Parameters In _executeRebalance	Best Practices	Info	Acknowledged
I-06	Unreachable Code In calculateCurrentRangeExcess	Best Practices	Info	Resolved
I-07	processRebalanceAfterWithdraw Function Is Unused	Best Practices	Info	Resolved
I-08	Only Last Overlapping Range Fixed In Rebalance	Documentation	Info	Acknowledged
I-09	Single-Step Ownership Transfer Risk	Best Practices	Info	Acknowledged

Findings & Resolutions

ID	Title	Category	Severity	Status
I-10	Unreachable Withdraw Flags Condition	Superfluous Code	Info	Resolved
I-11	Limit Positions Minted Without Slippage Checks	Warning	Info	Acknowledged
I-12	Compound Calls zeroBurn Twice Wasting Gas	Gas Optimization	Info	Acknowledged
I-13	Burn Event Emitted When Shares Are Not Burned	Events	Info	Acknowledged
I-14	Withdraw Sends ETH Before Burn	Reentrancy	Info	Resolved
I-15	Stale Position Storage After Full Withdrawal	Unexpected Behavior	Info	Resolved
I-16	Missing Events For Configuration Updates	Events	Info	Acknowledged
I-17	MIN_BALANCE Check Counts Msg.value	Unexpected Behavior	Info	Resolved
I-18	Missing Weight Validation In rebalanceSwap Path	Unexpected Behavior	Info	Resolved
I-19	Redundant Msg.value Check In deployRelayer	Best Practices	Info	Resolved
I-20	BURN_AND_REBALANCE Enum Name Is Misleading	Documentation	Info	Resolved
I-21	Mint May Allow Overpaying At Manipulated Price	Logical Error	Info	Acknowledged
I-22	Slippage Bypass On Single-Token Rebalance	MEV	Info	Acknowledged

Findings & Resolutions

ID	Title	Category	Severity	Status
I-23	withdrawCustom Missing Shares Slippage	Warning	Info	Acknowledged
I-24	Misleading StrategyParams Packing Comment	Documentation	Info	Resolved
I-25	Uint8 Loop Index Can Overflow	Warning	Info	Acknowledged
I-26	Zero Liquidity Positions Skip Compounding	Informational	Info	Acknowledged
I-27	Lens Preview Uses totalSupply For outMin	Informational	Info	Resolved
I-28	Native ETH Transfers Use Transfer	Best Practices	Info	Resolved
I-29	Asymmetric Limit Trigger At Lower Boundary	Suggestion	Info	Acknowledged

H-01 | Missing Deployer Access Control Causes DoS

Category	Severity	Location	Status
DoS	High	MultiPositionDeployer.sol: 26	Resolved

Description PoC

The `MultiPositionDeployer.deploy()` function is externally callable without access control. This function deploys `MultiPositionManager` contracts using `CREATE2` with a caller-supplied salt.

In the intended flow, `MultiPositionFactory.deployMultiPositionManager()` computes a deterministic salt and calls `MultiPositionDeployer.deploy()`. This flow is used by:

1. Token Launch Migration.

2. Pool Creation with Limit Order Support.

An attacker can extract parameters from the mempool or from deployed `SuperchainLBPStrategy` contracts (all fields are public). Using these parameters, the attacker computes the exact same salt and constructor arguments, then front-runs by calling `MultiPositionDeployer.deploy()` directly.

When the legitimate deployment proceeds, the factory attempts to deploy at the same address via the deployer, causing a `CREATE2` collision and reverting.

This will block migration and/or creating pools with limit order support.

Recommendation

Restrict `MultiPositionDeployer::deploy()` to only be callable by `MultiPositionFactory`.

Resolution

Gamma Team: Resolved.

M-01 | Unbounded Loop In deployMultiPositionManager

Category	Severity	Location	Status
DoS	Medium	MultiPositionFactory.sol: 179	Resolved

Description

The `deployMultiPositionManager` function loops through the `_allTokenPairs` array to locate and update a token pair's manager count.

As the number of unique token pairs grows, this linear scan becomes increasingly expensive and can lead to gas exhaustion or DoS, potentially blocking new deployments.

Recommendation

Add a hard cap or redesign the accounting so deployment cost doesn't grow with the number of pairs.

Resolution

Gamma Team: Resolved.

M-02 | RebalanceSwap Skips Protocol Fee Split

Category	Severity	Location	Status
Logical Error	Medium	MultiPositionManager.sol: 314	Resolved

Description

`MultiPositionManager.rebalanceSwap` unwinds all existing positions via `poolManager.unlock(Action.BURN_ALL, ...)`.

The `BURN_ALL` callback burns liquidity across positions and clears position state, but it does not execute the fee-splitting logic used elsewhere (the `zeroBurnAll` flow), where the treasury cut is computed as `totalFee / s.fee` and minted as ERC-6909 claims.

Because this unwind path skips that step, any accrued fees realized during the full burn are treated as ordinary vault assets and can be swapped/redeployed immediately, while the protocol fee recipient receives no claimable share.

Any caller permitted to call `rebalanceSwap` can bypass protocol fees by calling `rebalanceSwap` first and then proceeding with other actions.

Recommendation

Apply protocol fee accounting on all unwind paths. Run `zeroBurnAll` (or equivalent treasury-cut logic) before burning liquidity, or implement the treasury cut directly in the burn-all flow so fees are split consistently.

Resolution

Gamma Team: Resolved.

M-03 | ETH Accounting Breaks In Multicall

Category	Severity	Location	Status
Logical Error	Medium	MultiPositionManager.sol: 161	Resolved

Description

MultiPositionManager inherits Multicall and executes batches via delegatecall, meaning every subcall observes the same msg.value. For native-asset pools, deposit() ultimately relies on _transferIn(), which validates ETH payment using require(msg.value >= amount).

Because the contract does not track or decrement ETH across subcalls, multiple deposits in a single multicall can each independently satisfy the ETH check using the same msg.value.

Batching multiple deposits could be used when depositing for multiple recipients or when deposits are split across different from addresses/flows.

As a result, shares can be minted multiple times even though ETH was only provided once, inflating supply and diluting existing shareholders.

Depending on the deposit amounts and refund behavior, the contract may also attempt to incorrectly run refund logic multiple times within the same transaction, creating inconsistent ETH flows and potentially leaving the contract's ETH balance misaligned with assumptions in the share/accounting logic.

Recommendation

Disallow batching with ETH (revert if msg.value != 0 and data.length > 1), or implement an explicit ETH "remaining value" budget for multicall and consume it inside _transferIn() instead of reading msg.value per subcall.

Resolution

Gamma Team: Resolved.

M-04 | withdrawCustom Reverts On Fee Collection

Category	Severity	Location	Status
DoS	Medium	WithdrawLogic.sol: 275	Resolved

Description

WithdrawLogic.processWithdrawCustom can select a “balance + fees” path and calls zeroBurnAllWithoutUnlock(s, poolManager) directly. That function calls PoolManagerUtils.zeroBurnAll(), which invokes poolManager.modifyLiquidity() to realize fees.

In Uniswap v4, modifyLiquidity is expected to be executed while the PoolManager is unlocked (during poolManager.unlock() via the callback). Calling it directly from withdrawCustom outside an unlock is incompatible with standard v4 behaviour and can cause this branch to revert, making withdrawCustom fail whenever it selects the “balance + fees” path.

This can also impact automation. RelayerLogic.withdrawSingleToken (used by Relayer.executeWithdrawal) calls manager.withdrawCustom and may then call manager.rebalance to redeploy the remaining asset.

If withdrawCustom selects the USE_BALANCE_PLUS_FEES path, the relayer execution will revert due to fee collection occurring outside unlock, preventing the withdrawal and any subsequent rebalance in that transaction.

Recommendation

When withdrawCustom needs fee realization, execute it via poolManager.unlock(Action.ZERO_BURN, ...) (or via the existing Action.WITHDRAW callback flow) rather than calling zeroBurnAllWithoutUnlock directly.

Resolution

Gamma Team: Resolved.

M-05 | Zero Fee Can Brick Core Functions

Category	Severity	Location	Status
Math	Medium	MultiPositionManager.sol: 352	Resolved

Description

`setFee` in `MultiPositionManager` allows `s.fee` to be set to zero, with no minimum enforced. Several core execution paths assume `s.fee` is non-zero and divide by it to compute the protocol's share of accrued fees.

For example, `WithdrawLogic.getTotalAmounts()` adjusts TVL and fee totals using expressions like `totalFee / s.fee`, and fee-claim logic similarly derives mint/burn amounts from `totalFee / s.fee`. If `s.fee` is set to zero, these divisions revert.

As a result, setting `s.fee` to zero can cause withdrawals, fee claims, and other fee-dependent flows to revert, effectively bricking core functionality.

Recommendation

Enforce a non-zero minimum fee in `setFee`, or explicitly handle the zero-fee case in all accounting and fee-distribution logic to avoid division-by-zero reverts.

Resolution

Gamma Team: Resolved.

M-06 | Aggregator Address Not Validated

Category	Severity	Location	Status
Validation	Medium	RebalanceLogic.sol: 1385	Resolved

Description PoC

_executeAggregatorSwap is intended to prevent arbitrary external calls by restricting swaps to a approved set of aggregator integrations. It does this by validating params.aggregator (≤ 3), which corresponds to the supported options (ZERO_X, KYBERSWAP, ODOS, PARASWAP).

However, the actual call target, params.aggregatorAddress, is user-controlled and is not derived from or validated against the selected enum value. A caller can therefore pass a valid params.aggregator value while setting params.aggregatorAddress to an arbitrary contract.

Because _executeAggregatorSwap also grants params.aggregatorAddress an allowance for the input token (forceApprove) and then performs a low-level external call with user-controlled calldata, the enum validation does not prevent arbitrary external calls or malicious approval usage.

Any caller that can reach swap-based paths (including relayer/automation flows) can route execution to untrusted contracts and potentially drain or misdirect funds.

Even with a semi-trusted relayer, this is misleading: owners may grant relayer permissions believing swaps are constrained to known aggregators, but in practice the relayer can execute arbitrary external calls with token approvals because params.aggregatorAddress is not properly constrained.

Recommendation

Bind each aggregator enum value to a fixed, trusted aggregator address (or a whitelist per enum), and derive aggregatorAddress internally based on params.aggregator rather than accepting it as user input. Alternatively, explicitly validate that params.aggregatorAddress matches an address from an allowlisted set for the selected aggregator.

Resolution

Gamma Team: Resolved.

M-07 | ERC20 Shares Not Redeemable By Holders

Category	Severity	Location	Status
Logical Error	Medium	MultiPositionManager.sol	Partially Resolved

Description

MultiPositionManager issues transferable ERC20 shares, but redemption is tied only to owner instead of the shareholder.

Deposits can mint shares to an arbitrary to address. However, both withdrawal paths (withdraw and withdrawCustom functions) pay assets to and burn shares from owner.

Consequently, if shares are held by any address other than owner, those holders cannot redeem, and withdrawals may revert if owner address lacks the shares to burn.

Recommendation

Make shares non-transferable and always mint them to owner.

Resolution

Gamma Team: Partially Resolved.

M-08 | Rebalance Overflow When Price Near Lower Tick

Category	Severity	Location	Status
Logical Error	Medium	RebalanceLogic.sol: 605	Resolved

Description

In calculateCurrentRangeExcess, when the current price is close to the lower tick of a range, the denominator sqrtPriceX96 - sqrtPriceLower becomes small, producing an extremely large liquidity result that may exceed uint128.max and silently truncate.

Consider the following example (low priced pool):
sqrtPriceLower ≈ 1e18
sqrtPriceX96 ≈ 1.00005e18
denominator = sqrtPriceX96 - sqrtPriceLower ≈ 5e13
token1Allocation: 5e23 (18 decimals)
liquidityFrom1 = 5e23 * 7.9e28 / 5e13 = 7.9e38

However, uint128.max ≈ 3.4e38, therefore the result will silently truncate.

This will result in incorrect excess calculations and liquidity allocation across MPM positions. This may also cause DoS or unexpected behaviour to token launch migrations, depending on the launch and migration configuration.

Also consider applying the changes to mintFromAllocations .

Recommendation

Consider reverting upon overflow for both liquidityFrom1 and actualLiquidity calculations within calculateCurrentRangeExcess.

Resolution

Gamma Team: Resolved.

M-09 | Single-Token Withdraw DoS With Limit Positions

Category	Severity	Location	Status
DoS	Medium	RebalanceLogic.sol: 125	Resolved

Description

WithdrawSingleToken implemented in RelayerLogic, is used by the relayer withdrawal automation when configured for withdrawToken0Only or withdrawToken1Only.

The function first executes withdrawCustom to withdraw all amount of one token from the pool, then conditionally calls rebalance on MultiPositionManager to redeploy the remaining token into fresh positions.

The rebalance call is constructed with an outMin array sized only to the number of base positions. However, in RebalanceLogic, the rebalance function strictly requires the outMin length to match the number of currently active positions being burned (basePositionsLength + limitPositionsLength).

If the vault has any active limit positions, the call always reverts because withdrawSingleToken passes an outMin array that ignores limit positions. Consequently, single-token withdrawals will be permanently unusable for managers with active limit positions.

Recommendation

Size the rebalance outMin to cover all current positions - basePositionsLength + limitPositionsLength.

Resolution

Gamma Team: Resolved.

M-10 | Relayer Owner Desync From MPM Owner

Category	Severity	Location	Status
Configuration	Medium	Relayer.sol	Resolved

Description

Relayer admin control is permanently bound to the deploy-time manager owner via an immutable `owner` parameter, while the `MultiPositionManager` owner can change. All relayer admin functions use `onlyOwner` against this captured address.

However, execution already treats the MPM owner as dynamic:

```
address mpmOwner = Ownable(address(manager)).owner();
uint256 ownerShares = manager.balanceOf(mpmOwner);
manager.withdraw(ownerShares, outMin, true);
```

After MPM ownership is transferred, the new MPM owner cannot pause/configure/withdraw relayer ETH, while the old owner still can. Since the factory enforces a single relayer per manager, deploying a replacement is blocked, making this loss of control permanent.

Recommendation

Remove the captured immutable relayer owner and derive it from the current `MultiPositionManager` owner.

Resolution

Gamma Team: Resolved.

M-11 | Missing Token0 Fallback When liquidityFrom1 Is 0

Category	Severity	Location	Status
Logical Error	Medium	RebalanceLogic.sol: 634-639	Resolved

Description

In proportional mode (`weight0 == 0 && weight1 == 0`), `mintFromAllocations` computes liquidity starting from `token1`.

When `liquidityFrom1 = 0` (price at lower tick boundary, zero `token1` allocation after swaps, or one-sided deposits), the code never calculates liquidity from `token0` as a fallback. This causes `liquidities[i] = 0` even when sufficient `token0` exists.

The same issue exists in `calculateCurrentRangeExcess`, where `actualLiquidity` becomes 0 and all tokens are marked as excess and redistributed away from the current range.

With carpet mode enabled, `_validateCarpetLiquidity` requires non-zero liquidity for the outer carpet ranges and reverts if edge range liquidity is 0 (even with sufficient one-sided funds). This creates DoS when price lands exactly on a range boundary during initialization or rebalances.

Recommendation

Add fallback to calculate liquidity from `token0` when `token1` cannot contribute `calculateCurrentRangeExcess`.

Alternatively, replace the custom boundary handling with `LiquidityAmounts.getLiquidityForAmounts` which handles all boundary cases correctly.

Resolution

Gamma Team: Resolved.

M-12 | Relayer Off-By-One Ratio Extraction

Category	Severity	Location	Status
Logical Error	Medium	Relayer.sol: 382	Resolved

Description

In Relayer contract, getRatios returns inPositionRatio and outOfPositionRatio as respectively 5th and 6th values. However, Relayer unpacks the returned values as if outOfPositionRatio were the 5th return value.

```
(,,,, uint256 outOfPositionRatio,,,,,,,) = manager.getRatios();
```

This binds the 5th return value (inPositionRatio) to a variable named outOfPositionRatio. That means the relayer checks the wrong condition and it effectively allows compounding when assets are in position (inPositionRatio >= threshold) instead of when assets are idle (outOfPositionRatio >= threshold).

In practice, this makes executeCompoundSwap behave like it is almost always enabled even when the vault is not meaningfully out of position, wasting ETH on reimbursements and potentially causing unnecessary swaps

Recommendation

Update Relayer's getRatios unpacking to use the correct outOfPositionRatio position.

Resolution

Gamma Team: Resolved.

M-13 | Unaligned TWAP Center Tick

Category	Severity	Location	Status
Logical Error	Medium	RelayerLogic.sol	Resolved

Description

When TWAP-based centering is used (`baseTwapTickTrigger` or `useTwapCenter`), the relayer sets params.center to the raw TWAP tick returned by the oracle. That TWAP tick is not guaranteed to be aligned to the pool's `tickSpacing`.

Range generation does not use that raw value directly. For example, `SingleUniformStrategy` snaps the received `centerTick` onto the `tickSpacing` grid using `centerTick = (centerTick / tickSpacing) * tickSpacing`.

This uses truncating division, so the "center" actually used to build ranges can differ from the value stored in `lastStrategyParams.centerTick` whenever the TWAP tick is not already aligned.

This mismatch is more severe for negative ticks because truncation rounds toward zero (e.g., `tickSpacing = 60, TWAP = -1` snaps to 0 instead of -60).

Any subsequent logic that references the stored center (for example, trigger calculations or comparisons relative to `centerTick`) can become inconsistent because the system records one center while liquidity was deployed around a different one.

Recommendation

Align the TWAP tick to `tickSpacing` before using it as `params.center`, using the same rounding convention as the manager's tick alignment (round down for negatives), and store that aligned value as `lastStrategyParams.centerTick`. Furthermore, update strategies such as `SingleUniformStrategy` to align ticks using floor-style rounding for negative ticks rather than truncating division.

Resolution

Gamma Team: Resolved.

M-14 | Lens Ignores Fee Claim Before Rebalance

Category	Severity	Location	Status
Logical Error	Medium	Relayer.sol: 330-332	Resolved

Description

When compoundFees is false, the relayer always calls claimFee on MPM before rebalancing, and triggers WithdrawLogic.processClaimFee, which zero-burns fees, then pays the owner's share out of the MPM and forwards the treasury share to the factory feeRecipient.

Consequently, those transfers remove fee amounts from the manager's token balances immediately before the rebalance.

However, in RelayerLens, previewRebalance ignores this step and bases expected positions, swapAmount and inMin on getTotalAmounts that still include the unpaid fees.

With non-zero fees, the preview overstates available tokens and automation can submit a rebalance that reverts once fees are paid, despite the lens reporting it should succeed, causing failed automation and wasted gas.

Recommendation

Update previewRebalance to simulate claimFee when compoundFees is false by using balances after the claim.

Resolution

Gamma Team: Resolved.

M-15 | Slippage Bypass When Limit Position 0 Empty

Category	Severity	Location	Status
Logical Error	Medium	PositionLogic.sol: 100-111,,364-375	Resolved

Description PoC

During rebalancing, the limit positions are updated only if the positions are non-empty.

This allows a state where `limitPositions[0]` is empty while `limitPositions[1]` is non-empty, resulting in `limitPositionsLength = 1`.

However, when burning limit positions, the `outMin` index is derived from the slot index i.

Applying slippage will not be possible if the `outMin` list is limited only to the length of `s.basePositionsLength + s.limitPositionsLength` in this case, forcing the owner to remove the limit position liquidity with 0 slippage.

Consider the following example:
- `basePositionsLength = 3`
- `limitPositions[0]` is empty (0, 0)
- `limitPositions[1]` is non-empty (100, 200)
- `limitPositionsLength = 1` (only position 1 counted)

Validation requires `outMin.length = basePositionsLength + limitPositionsLength = 3 + 1 = 4` User provides `outMin` with indices [0, 1, 2, 3].

During `burnLimitPositions`:
i = 0: `limitRanges[0]` is empty, so this iteration is skipped.
i = 1: `limitRanges[1]` is non-empty, so `outMinIndex` = baseRangesLength + i = 3 + 1 = 4.
Check: 4 < 4 → False.
Falls back to [0, 0] → zero slippage.

This will remove liquidity from the position with 0 slippage, with MPM owners having no way around this, leading to loss of funds (i.e sandwich attack, price movement, etc).

There is another issue in a separate function, called during compounds, `DepositLogic::addLiquidityToPositions`. The `for (uint8 i = 0; i < limitLength;) {` loop will skip adding liquidity to `limitPosition[1]` if `limitPosition[0]` is empty, causing yield loss (same root cause).

Recommendation

Consider utilizing a limit counter to properly fetch the correct index.

Resolution

Gamma Team: Resolved.

M-16 | Rebalance Slippage Ignores Principal Vs Fees

Category	Severity	Location	Status
Error	Medium	PoolManagerUtils.sol: 271-291	Resolved

Description

When burning liquidity positions during rebalancing, `outMin` slippage checks are performed against `callerDelta` which includes both principal and accrued fees.

Uniswap V4's `PositionManager` explicitly subtracts fees before slippage validation because `minOut` parameters are intended to protect the principal value.

There are two cases during rebalances that are affected:

1.`rebalanceSwap()`: Uses `BURN_ALL` without collecting fees prior.

2. `rebalance()` with only limit positions. The `ZERO_BURN` check only considers base positions.

Because accrued fees are included in the slippage comparison, they can mask losses on the principal portion.

Due to price changes or manipulation before the owner's rebalance transaction is executed, the principal value can be reduced, while the slippage check still passes due to accumulated fees acting as a buffer.

Recommendation

For `rebalanceSwap()`: Either follow Uniswap V4 `PositionManager`'s pattern by subtracting `feesAccrued` from `callerDelta` before checking against `outMin`, or collect fees prior to burning positions (similar to the withdrawal flow).

For `rebalance()`: Update the `ZERO_BURN` condition to also check for limit positions (`s.basePositionsLength > 0 || s.limitPositionsLength > 0`).

Resolution

Gamma Team: Resolved.

M-17 | Gas Refund Ignores L1 Data Fee

Category	Severity	Location	Status
Unexpected Behavior	Medium	Relayer.sol: 742	Partially Resolved

Description

_reimburseGas refunds only the L2 execution cost using the following calculations:

```
// Calculate gas used
uint256 gasUsed = gasBefore - gasleft() + BASE_GAS_OVERHEAD;
// Calculate reimbursement with 10% buffer
uint256 reimbursement = (gasUsed * tx.gasprice * GAS_BUFFER_NUMERATOR) /
GAS_BUFFER_DENOMINATOR;
```

However, Unichain (where the protocol is intended to be deployed) is an Ethereum L2 built on the OP Stack.

On OP Stack chains, sequenced L2 transactions are charged an additional L1 data fee that covers publishing the transaction batch data to Ethereum, which is not captured by gasUsed * tx.gasprice.

This means automation services will be systematically under-reimbursed, and the gap can be especially large for calls with large calldata

Recommendation

Reimburse the full transaction cost by adding the L1 data fee.

Resolution

Gamma Team: Partially Resolved.

M-18 | limitPositionsLength Can Skip Active Limit Slot

Category	Severity	Location	Status
Logical Error	Medium	PositionLogic.sol: 55	Resolved

Description

The `MultiPositionManager` stores limit positions in two fixed slots, `s.limitPositions[0]` for the lower range and `s.limitPositions[1]` for the upper range. The variable `s.limitPositionsLength` is intended to represent how many of these slots are active.

Several code paths iterate limit positions using `for (i < s.limitPositionsLength)` and then access `s.limitPositions[i]`. This logic assumes that when exactly one limit position is active, it must be stored at index `0`. That assumption is not guaranteed.

If the lower limit position collapses into an empty range due to tick rounding or clamping near the minimum usable tick while the upper limit remains valid, `s.limitPositionsLength` becomes `1` even though the only active position resides at index `1`. In this case, loops bounded by `s.limitPositionsLength` will only read `s.limitPositions[0]` and will skip the active upper limit.

As a result, deployed liquidity and accrued fees can be undercounted in aggregation logic such as total amount calculations. Liquidity add or rebalance flows may also ignore an active limit range, leading to incorrect allocations, understated vault value, or unexpected idle balances.

Recommendation

Do not use `s.limitPositionsLength` as an index bound for accessing `s.limitPositions`. Always iterate over both fixed slots and skip inactive entries by checking that `lowerTick != upperTick`.

Resolution

Gamma Team: Resolved.

M-19 | Carpet Blocks Relayer Single-Token Withdraw

Category	Severity	Location	Status
Logical Error	Medium	RebalanceLogic.sol: 305	Resolved

Description

When a withdrawal trigger is configured with `withdrawToken0Only` or `withdrawToken1Only`, the relayer calls `RelayerLogic.withdrawSingleToken`, which first withdraws all of the specified token using `withdrawCustom`, and then attempts to rebalance the remaining assets.

Then, the relayer fetches the last strategy parameters and calls `rebalance` on MPM. using `lastUseCarpet` parameter.

However, the rebalance implementation enforces that carpet mode requires both tokens to be non-zero, reverting when either available token amount is zero.

```
if (ctx.useCarpet && (available0 == 0 || available1 == 0)) {
revert CarpetRequiresBothTokens();
}
```

Since the single-token withdrawal intentionally makes one token amount zero, the subsequent rebalance reverts whenever `lastUseCarpet` is true, causing the entire automated withdrawal transaction to revert.

Recommendation

Either change `withdrawSingleToken` to not attempt a carpet rebalance after making the vault one-sided or explicitly disallow
configuring `withdrawToken0Only`/`withdrawToken1Only` when `useCarpet` is enabled.

Resolution

Gamma Team: The issue was resolved in commit af04adb.

M-20 | Proportional Mode Handled Inconsistently

Category	Severity	Location	Status
Logical Error	Medium	MultiPositionManager.sol: 314	Resolved

Description

The protocol treats proportional mode (`weight0 == 0 && weight1 == 0`) differently depending on the rebalance entrypoint.

In the `rebalance()` flow, `_processRebalance()` explicitly disables limit logic by forcing `limitWidth = 0`, stating that limit positions do not make sense when weights are derived from amounts. This guarantees that proportional rebalances only construct base ranges.

However, the `rebalanceSwap()` flow does not apply the same restriction. When building the strategy context, `_buildStrategyContext()` forwards `params.limitWidth` unchanged even if `weight0 == 0 && weight1 == 0`. As a result, `rebalanceSwap()` can execute in proportional mode with a nonzero limit width, allowing strategies to construct limit positions.

This leads to inconsistent behaviour where two operations that appear equivalent can result in different final positions depending on the entrypoint used. In particular, operators assumptions that proportional mode always results in base-only positions may break if a rebalance is executed through `rebalanceSwap()` instead.

Recommendation

Normalize proportional mode handling by setting `limitWidth = 0` whenever `weight0 == 0` & `weight1 == 0` in the `rebalanceSwap()` path as well, or clearly document that proportional mode may include limit position construction.

Resolution

Gamma Team: Resolved.

M-21 | rebalanceSwap Bypasses Carpet Checks

Category	Severity	Location	Status
Unexpected Behavior	Medium	RebalanceLogic.sol: 305-307	Resolved

Description

The standard rebalance path enforces carpet invariants when `useCarpet` is set to true. It requires both tokens to be non-zero and it the edge carpet positions to have non-zero liquidity.

The `rebalanceSwap` path does not execute this validation logic. It burns positions, optionally swaps and persists `lastStrategyParams.useCarpet=true`, without validating carpet liquidity and total available amounts.

As a result, `rebalanceSwap` can persist `useCarpet` set to true, even when the post-swap state is one-sided or assigns zero liquidity to the carpet edges, violating the intended carpet mechanism.

Recommendation

Apply the same carpet checks in the `rebalanceSwap` path.

Resolution

Gamma Team: The issue was resolved in commit af04adb.

M-22 | Unreachable Single-Token Withdrawal Rebalance

Category	Severity	Location	Status
Logical Error	Medium	RelayerLogic.sol: 387-393	Resolved

Description

The withdrawal path is initiated in executeWithdrawal by checking in-position ratios returned by getRatios and then calling withdrawSingleToken that is supposed to withdraw token0 or token1 and rebalance with the other token.

This function withdraws the entire selected token amount using getTotalAmounts, which includes position amounts, fees and idle balances.

When the relayer trigger for “withdraw only token0” is met, pool0Ratio must be non-zero (it is computed from in-position amounts). In that state, total0 necessarily includes some token0 that is currently inside positions.

Therefore, withdrawCustom cannot be satisfied from idle balances alone and must enter the burn positions path, which computes how many shares worth of positions must be burned to source the requested token amount.

Because amount0Desired is equal to total0 in this flow, sharesForToken0 becomes totalSupply, so it burns 100% of positions. After positions are fully burned, the in-position amounts are zero, so getRatios reports pool0Ratio == 0 and pool1Ratio == 0 regardless of any remaining idle token balances (as idle balances are not included in pool0Ratio/pool1Ratio).

This makes the subsequent rebalance block unreachable in the intended, triggered case, even if the burn left a non-zero amount of the other token sitting idle in the manager, what consequently break the intended “withdraw and rebalance” behaviour.

Also, the only scenario where the rebalance block could run is when the selected token is entirely idle (so no burn happens and positions still contain the other token), but in that scenario the relayer cannot trigger “withdraw token0 only” or “withdraw token1 only” because the corresponding in-position ratio is zero and cannot meet a non-zero threshold.

Recommendation

Change the post withdraw rebalance check to use a value that includes idle balances rather than pool0Ratio/pool1Ratio which reflect positions only.

Resolution

Gamma Team: Resolved.

L-01 | Relayer Deployment Griefing

Category	Severity	Location	Status
DoS	Low	RelayerDeployer.sol: 27	Resolved

Description

Relayer deployment relies on `CREATE2` to deterministically compute relayer addresses from the MPM address, owner address, and configuration parameters.

While `RelayerFactory` enforces authorization checks, the underlying `RelayerDeployer.deploy()` function itself is permissionless and can be called directly.

As a result, any user can front-run a legitimate deployment by calling `RelayerDeployer.deploy()` with the same parameters the owner intends to use. This preemptive deployment occupies the expected relayer address.

When the rightful MPM owner later attempts to deploy through the factory, the call reverts because the contract is already deployed.

This enables a griefing attack where attackers can block MPM owners from deploying relayers with their intended configuration.

Recommendation

`Restrict RelayerDeployer.deploy()` so it can only be called by the authorized factory, ensuring relayer creation cannot be front-run or griefed by external callers.

Resolution

Gamma Team: Resolved.

L-02 | Deposit Function Lacks Slippage Protection

Category	Severity	Location	Status
Error	Low	MultiPositionManager.sol: 161	Acknowledged

Description

The `deposit()` function calculates shares using the `slot0` price without any slippage protection parameter (`minSharesOut`).

While the owner is typically the sole shareholder and cannot lose value to themselves, in edge cases where the owner mints shares to third-party addresses via the `to` parameter, or in future integrations, this could lead to share recipients receiving fewer shares than expected due to price movements or manipulation (i.e., sandwich attack).

Recommendation

Add `minSharesOut` parameter to `deposit()` for slippage protection, and consider validating `sqrtPriceX96` against a slippage `threshold`.

Resolution

Gamma Team: Acknowledged.

L-03 | TWAP Validation Mismatch After Deployment

Category	Severity	Location	Status
Configuration	Low	Relayer.sol: 152	Resolved

Description

Relayer deployment and post deploy updates enforce different TWAP constraints. `RelayerFactory` rejects overly large TWAP windows and verifies the oracle can serve the requested `twapSeconds`.

After deployment, the relayer owner can call `setRebalanceParams`, which uses `_validateTwapParams` and does not enforce the same `twapSeconds` upper bound or re-check oracle availability.

If `twapSeconds` is set beyond the oracle's retained history, `oracle.consult` can revert and brick all paths that rely on TWAP until the params are corrected.

Recommendation

Consider enforcing the same TWAP bounds and oracle availability validation in `Relayer._validateTwapParams` as in the factory.

Resolution

Gamma Team: Resolved.

L-04 | useRebalanceSwap Not Enforced On-Chain

Category	Severity	Location	Status
Unexpected Behavior	Low	Relayer.sol	Resolved

Description

In `StrategyParams` struct, `useRebalanceSwap` parameter is treated as the configuration setting that indicates whether rebalances should use the swap path, but the relayer does not enforce it at execution time.

A whitelisted automation service can always call either `executeRebalance` or `executeRebalanceSwap`, regardless of the configured flag.

Off-chain preview contract (`RelayerLens`) relies on `useRebalanceSwap` to decide whether a swap is expected, but on-chain the flag is not enforced at execution time.

If the automation service is misconfigured, it can execute the unintended path (swap when disabled or no-swap when swap is required), potentially leading to unnecessary and unexpected executions.

Recommendation

Consider gating `executeRebalance` and `executeRebalanceSwap` so the callable entrypoint must match `useRebalanceSwap` parameter, making the configuration enforceable on-chain.

Resolution

Gamma Team: Resolved.

L-05 | Incomplete Limit Width Collision Check

Category	Severity	Location	Status
Error	Low	PositionLogic.sol: 73-84	Resolved

Description

In setLimitRanges, when a base range width matches limitWidth, the code increments limitWidth once and immediately exits the loop. The new value is not rechecked against remaining base ranges, allowing another collision to remain undetected.

```
for (uint256 i = 0; i < baseRangesLength;) {
int24 rangeWidth = baseRanges[i].upperTick - baseRanges[i].lowerTick;
if (rangeWidth == int24(limitWidth)) {
limitWidth = uint24(int24(limitWidth) + tickSpacing);
break;  // Exits without checking the rest
}
unchecked { ++i; }
}
```

If the resulting limit range later matches a base range's (lowerTick, upperTick), checkRanges() will revert with DuplicatedRange, causing the rebalance or migration to fail.

Recommendation

Consider iterating until limitWidth no longer collides with any base range width.

Resolution

Gamma Team: Resolved.

L-06 | TWAP Protection Gas Not Reimbursed

Category	Severity	Location	Status
Logical Error	Low	Relayer.sol: 316	Resolved

Description

executeRebalance and executeRebalanceSwap run _checkTwapProtection before sampling gasleft.

Since _reimburseGas only accounts for gas spent after gasBefore is captured, the oracle/TWAP check gas is excluded from reimbursement even on successful executions.

Consequently, automation service is systematically under-reimbursed when TWAP protection is enabled.

Recommendation

Capture gasBefore before calling _checkTwapProtection.

Resolution

Gamma Team: Resolved.

L-07 | Native Currency Breaks getUniqueTokenPairs()

Category	Severity	Location	Status
Logical Error	Low	RelayerFactory.sol: 406	Resolved

Description

RelayerFactory.getUniqueTokenPairs() iterates through deployed relayers and reads each relayer's poolKey to derive token metadata (symbol and decimals).

It unwraps each Currency into an address and treats it as an ERC-20 by calling IERC20Metadata(token).symbol() and IERC20Metadata(token).decimals().

For Uniswap v4 pools that use the native currency (ETH), the corresponding Currency unwraps to address(0). As a result, getUniqueTokenPairs() will call IERC20Metadata(address(0)), which will revert.

Recommendation

Handle native currency explicitly before calling ERC-20 metadata methods. If token == address(0), return a fixed symbol ("ETH") and decimals (18).

Resolution

Gamma Team: Resolved.

L-08 | Proportional Rebalance Can Revert At Min Price

Category	Severity	Location	Status
DoS	Low	RebalanceLogic.sol: 768-772	Resolved

Description

In proportional mode (`weight0 == 0 && weight1 == 0`), the current range math computes `token0Needed` using the return value of the following equation, as a denominator:

```
FullMath.mulDiv(sqrtPriceUpper, sqrtPriceX96, FixedPoint96.Q96)
```

Near extreme `MIN_SQRT_PRICE` (2^32), this inner `mulDiv` can floor to 0 (when `sqrtPriceUpper * sqrtPriceX96 < 2^96`), causing the outer `mulDiv` to revert due to division by zero.

This is only reachable at very extreme pool prices, but it is still a valid Uniswap state. In this scenario, proportional rebalances can revert, halting automated rebalancing and compounding at those prices.

Recommendation

Be aware of this edge case and add a simple guard before the outer `mulDiv` so proportional rebalances never revert due to division by zero.

Resolution

Gamma Team: Resolved.

L-09 | User-Specified Weights Silently Overridden

Category	Severity	Location	Status
Logical Error	Low	RebalanceLogic.sol: 260-263	Resolved

Description

When a user provides explicit weights (e.g., weight0=0.7e18, weight1=0.3e18), the system validates they sum to 1e18 but may silently override them to 50/50 in `calculateWeightsWithPoolKey` if the strategy doesn't support weighted distribution:

```
if (!params.useCarpet && !supportsWeightedDist && (params.weight0 != 0.5e18 || params.weight1
!= 0.5e18)) {
params.weight0 = 0.5e18;
params.weight1 = 0.5e18;
}
```

The ctx struct is never updated to reflect this change. Subsequently, `_calculateLiquiditiesFromWeights` operates on weights calculated with 50/50 weighting, causing a mismatch between user intent and actual execution.

A user requesting 70/30 may get silently ignored, and would instead get 50/50 weighting from the strategy's density computation, leaving idle token/unused balances.

Recommendation

Consider reverting when weights are requested (explicit) but unsupported.

Resolution

Gamma Team: Resolved.

L-10 | ExactOut Swaps Assume Full Input Consumed

Category	Severity	Location	Status
Logical Error	Low	RebalanceLogic.sol: 1378-1436	Acknowledged

Description

In _executeProvidedSwap, the code assumes the full swapAmount is consumed by the aggregator:

```
if (swapParams.swapToken0) {
return (amount0 - swapParams.swapAmount, amount1 + amountOut);
}
return (amount0 + amountOut, amount1 - swapParams.swapAmount);
```

For exactOut swaps, the aggregator may consume less than swapAmount (which represents maximum input). The code subtracts the full swapAmount regardless, underestimating the remaining input token balance.

In the rebalanceSwap flow, this causes liquidity calculations based on understated balances resulting in under-minting of liquidity positions. The difference remains idle in the contract.

Recommendation

Track actual input spent by checking balance difference before and after swap (for ETH and ERC20).

Resolution

Gamma Team: Acknowledged.

L-11 | Dust Pricing Can Brick withdrawCustom

Category	Severity	Location	Status
DoS	Low	WithdrawLogic.sol: 365	Resolved

Description

The withdrawCustom flow relies on converting the vault's total value and the requested withdrawal value into token1 terms, then computing how many shares to burn.

This has two related failure modes caused by integer rounding:

• poolValueInToken1 can become 0 even though the vault holds assets. This happens when the vault is one-sided in token0 (so pool1 == 0) and the conversion FullMath.mulDiv(pool0, price, PRECISION) rounds down to 0 because the entire token0 balance is worth less than 1 smallest unit of token1.

• even when poolValueInToken1 is nonzero, the same rounding behavior can make withdrawalValue0InToken1 round down to 0 for small token0-only withdrawals, which can produce shares == 0.

Consequently, withdrawCustom can be DoS'd for valid, edge-case states.

Recommendation

Be aware of these edge cases in calculateSharesToBurn and add explicit guards/fallbacks so poolValueInToken1 can't be 0 and any nonzero withdrawCustom burns at least 1 share.

Resolution

Gamma Team: Resolved.

L-12 | Relayer Deploy Lacks Ratio Validation

Category	Severity	Location	Status
Configuration	Low	Relayer.sol: 85	Resolved

Description

Relayer is intended to be deployed via `RelayerFactory`, but neither the factory nor the Relayer constructor validates the ratio fields in `TriggerConfig`.

The constructor only validates deltas and weights before storing `_triggerConfig`, and `_validateRatios` is only enforced later in `setRebalanceParams`.

This means a relayer can be deployed through the factory with out-of-range ratios (>1e18) or `baseMinRatio > baseMaxRatio`, resulting in permanently misconfigured triggers until the owner updates parameters.

The deployment path therefore allows invalid trigger thresholds that the update path would reject.

Recommendation

Validate `TriggerConfig` ratios at deployment by calling `_validateRatios` in the constructor or factory.

Resolution

Gamma Team: Resolved.

L-13 | Relayer Address Computation Uses Arbitrary Owner

Category	Severity	Location	Status
Logical Error	Low	RelayerFactory.sol: 255	Resolved

Description

`computeRelayerAddress` accepts an arbitrary `owner` parameter and uses it in the `CREATE2` address computation, but `deployRelayer` ignores any external owner input and always deploys the relayer with the actual MPM owner.

If a caller computes the address with any owner other than the MPM owner, the computed address will not match the deployed address and consequently, external integrations can end up configured with the wrong address.

Recommendation

Consider removing the `owner` parameter and always use `MultiPositionManager(mpm).owner` for address computation.

Resolution

Gamma Team: Resolved.

L-14 | Owner Can Block Protocol claimFee For ETH Pools

Category	Severity	Location	Status
DoS	Low	MultiPositionManager.sol: 337	Resolved

Description

When MultiPositionManager::claimFee is executed by the factory CLAIM_MANAGER, address(0) is encoded to skip owner fee collection. However, an MPM owner can call grantRelayerRole(CLAIM_MANAGER), causing the CLAIM_MANAGER to match the first condition:

```
function claimFee() external {
if (msg.sender == owner() || s.relayers[msg.sender]) {
poolManager.unlock(abi.encode(IMultiPositionManager.Action.CLAIM_FEE, abi.encode(owner())));
}
```

This will attempt to send ETH fees to the owner, triggering the receive() function. A malicious owner can revert this transaction, thus blocking protocol treasury fee collection.

Eventually, the owner must withdraw and claim fees, which will allow fee collection by skipping owner ETH fee collection.

However, if the owner decides to burn up to 99% of their positions, and follow-up with a small donation via PoolManager::donate, this will accrue fees in the existing in-range positions, thus blocking protocol fee collection permanently (as it will always attempt to send small ETH fees to the owner).

Recommendation

Do not allow MPM owners to grant relayer role to the CLAIM_MANAGER address.

Resolution

Gamma Team: Resolved.

L-15 | Owner Can Blacklist Protocol Fee Recipient

Category	Severity	Location	Status
DoS	Low	WithdrawLogic.sol: 575-576	Acknowledged

Description

When fees are collected via `MultiPositionManager::claimFee`, both `token0` and `token1` fees are collected together:

```
_claimFeeCurrency(poolManager, s.factory, s.currency0);
_claimFeeCurrency(poolManager, s.factory, s.currency1);
```

If any of these calls fail, this blocks fee collection for both `currency0` and `currency1`. Given the design of the protocol is to facilitate token launches, any custom token can contain blacklist mechanisms.

If a malicious owner blacklists the protocol fee recipient, the `claimFee` transaction will revert, and the protocol will also lose fees on the other currency (which, given the auction mechanism, in most cases will be ETH/USDC).

Recommendation

Consider separating the fee collection for both tokens in two different functions, so if fee collection fails for one token, it can still be possible to collect for the other token.

Resolution

Gamma Team: Acknowledged.

L-16 | Carpet Mode Reverts On Small Deposits

Category	Severity	Location	Status
DoS	● Low	RebalanceLogic.sol: 797	Resolved

Description

Carpet positions use a fixed weight of 0.005% (CARPET_WEIGHT = 0.00005e18) and span the full tick range (min to max usable ticks). When calculating liquidity:

```
liquidity = token1 * Q96 / (sqrtPriceUpper - sqrtPriceLower)
```

The extreme tick range can create a large denominator, which combined with the relatively small fixed allocation, liquidity rounds down to zero for small deposit amounts. The validation then reverts:

```
function _validateCarpetLiquidity(...) {
if (baseRanges[0].lowerTick == minUsable && liquidities[0] == 0) {
revert InsufficientLiquidityForCarpet();
}
}
```

For regular rebalances, users can disable carpet mode. However, during migration via LBPStrategyBasic, if currencyRaised from the auction is small and carpet mode is enabled, migration will be permanently blocked since parameters are preset at deployment.

Recommendation

Similar to base positions, skip carpet positions gracefully when liquidity rounds to zero instead of reverting. Alternatively, allow users to disable carpet mode during migration if needed.

Resolution

Gamma Team: Resolved.

L-17 | MPM Fee Revenue Bypassed Via Custom Hook Pools

Category	Severity	Location	Status
Compatibility	Low	MultiPositionFactory.sol: 126	Acknowledged

Description

The protocol's revenue model relies on splitting LP fees collected via `zeroBurnAll` between the position owner and the fee recipient.

However, Uniswap V4 hooks can capture swap fees before they accrue to LP positions through mechanisms like `BEFORE_SWAP_RETURNS_DELTA` or LP fee overrides.

Since MPM allows creating positions on any pool, users can select pools with fee-capturing hooks. In such pools, LP positions accrue reduced or zero fees, leaving the protocol with no revenue despite providing a managed liquidity service.

Recommendation

Consider validating pool hooks during position creation, or implementing an alternative fee mechanism that doesn't depend solely on LP fee accrual.

Resolution

Gamma Team: Acknowledged.

I-01 | Unused ETH Causes Revert On Deposits

Category	Severity	Location	Status
DoS	● Info	MultiPositionManager.sol: 488	Acknowledged

Description

When creating pools through OrderBookFactory, the desired deposits and ethForDeployment is calculated, followed by MGM deployment through either MultiPositionFactory.deployDepositAndRebalance() or deployDepositAndRebalanceSwap().

In case a user sends ETH exceeding the deposit desired, it is refunded to msg.sender: MultiPositionManager::_transferIn:

```
function _transferIn(address from, Currency currency, uint256 amount) internal {
if (currency.isAddressZero()) {
require(msg.value >= amount);
if (msg.value > amount) {
payable(msg.sender).transfer(msg.value - amount);
}
...
}
```

However, in the OrderBookFactory or Token Launch Migration flow, msg.sender is the MultiPositionFactory contract, which lacks a receive() function.

This means any transaction with refunded ETH will revert, causing DoS to pool launches with limit order support.

Recommendation

Add a receive() function to MultiPositionFactory and And refund excess ETH to the original caller at the end of deployDepositAndRebalance() and deployDepositAndRebalanceSwap().

Resolution

Gamma Team: Acknowledged.

I-02 | Missing Deadline Parameter On MPM Functions

Category	Severity	Location	Status
Validation	Info	MultiPositionManager.sol: 161-330	Acknowledged

Description

Multiple functions in `MultiPositionManager` lack deadline parameters, including `deposit()`, `withdraw()`, `withdrawCustom()`, `rebalance()`, and `compound()`.

Without deadlines, transactions can remain pending in the mempool and execute at a much later time when market conditions have changed significantly, potentially resulting in unfavorable outcomes for the user (i.e., during deposits, users may receive shares at an outdated price).

Recommendation

Add a deadline parameter to the above MPM functions

Resolution

Gamma Team: Acknowledged.

I-03 | Dead Code In scaleAllocations Function

Category	Severity	Location	Status
Best Practices	Info	RebalanceLogic.sol: 497	Acknowledged

Description

The `scaleAllocations` function in `RebalanceLogic` contains an unreachable else branch that handles explicit weights mode (`useAssetWeights = false`).

This code path can never be executed because the only caller, `_calculateLiquiditiesFromWeights` always calls this function with `useAssetWeights = true`, and returns early when explicit weights are used.

Recommendation

Remove the `else` branch and the `useAssetWeights` parameter from `scaleAllocations`.

Resolution

Gamma Team: Acknowledged.

I-04 | Redundant Loop In scaleAllocations

Category	Severity	Location	Status
Gas Optimization	Info	RebalanceLogic.sol: 508-524	Acknowledged

Description

The scaleAllocations function in RebalanceLogic iterates over the same array indices twice (once for token0Allocations and once for token1Allocations). Since both arrays are always the same length, these loops can be combined into a single iteration to save gas.

Recommendation

Combine the loop into one to save gas:

```
for (uint256 i = 0; i < rangesLength;) {
if (data.totalToken0Needed != 0) {
data.token0Allocations[i] = FullMath.mulDiv(data.token0Allocations[i], available0,
data.totalToken0Needed);
}
if (data.totalToken1Needed != 0) {
data.token1Allocations[i] = FullMath.mulDiv(data.token1Allocations[i], available1,
data.totalToken1Needed);
}
unchecked { ++i; }
}
```

Resolution

Gamma Team: Acknowledged.

I-05 | Unused Slippage Parameters In _executeRebalance

Category	Severity	Location	Status
Best Practices	Info	RebalanceLogic.sol: 287-288	Acknowledged

Description

The `_executeRebalance` function accepts `outMin` and `inMin` parameters but never uses them:

```
function _executeRebalance(
SharedStructs.ManagerStorage storage s,
IPoolManager poolManager,
StrategyContext memory ctx,
IMultiPositionManager.Range[] memory baseRanges,
uint256[] memory weights,
uint256[2][] memory, /* outMin */  // Unused
uint256[2][] memory /* inMin */    // Unused
)
```

The actual slippage protection occurs later in `processRebalanceInCallback`.

Recommendation

Remove the unused parameters from `_executeRebalance` signature to improve code clarity and save gas.

Resolution

Gamma Team: Acknowledged.

I-06 | Unreachable Code In calculateCurrentRangeExcess

Category	Severity	Location	Status
Best Practices	Info	RebalanceLogic.sol: 629-631	Resolved

Description

In calculateCurrentRangeExcess, the else branch setting actualLiquidity = 0 is unreachable:

```
if (sqrtPriceX96 < sqrtPriceUpper) {
uint256 intermediate = FullMath.mulDiv(sqrtPriceUpper, sqrtPriceX96, FixedPoint96.Q96);
actualLiquidity =
uint128(FullMath.mulDiv(data.token0Allocations[idx], intermediate, sqrtPriceUpper -
sqrtPriceX96));
} else {
actualLiquidity = 0;  // Unreachable
}
```

If token0Needed is non-zero, then that already means sqrtPriceX96 < sqrtPriceUpper (due to the previous lines). Therefore, for the if (data.token0Allocations[idx] < token0Needed) branch to execute, if (sqrtPriceX96 < sqrtPriceUpper) will always be true, thus never entering the else branch.

The unreachable branch also exists in mintFromAllocations for the current range branch.

Recommendation

Remove the unreachable else branch.

Resolution

Gamma Team: Resolved.

I-07 | processRebalanceAfterWithdraw Function Is Unused

Category	Severity	Location	Status
Best Practices	Info	RebalanceLogic.sol: 1021	Resolved

Description

`RebalanceLogic.processRebalanceAfterWithdraw` is implemented but never called. This function is intended for automatic rebalancing after single token withdrawals.

Note that owners can achieve the same result by manually calling `rebalance()` after withdrawing.

Recommendation

Remove the dead code or integrate into withdrawal flow.

Resolution

Gamma Team: Resolved.

I-08 | Only Last Overlapping Range Fixed In Rebalance

Category	Severity	Location	Status
Documentation	Info	RebalanceLogic.sol: 480-484	Acknowledged

Description

In RebalanceLogic::calculateInitialAllocations, when multiple position ranges contain the current tick, only the last one is recorded as currentRangeIndex:

```
for (uint256 i = 0; i < rangesLength;) {
...
if (baseRanges[i].lowerTick <= data.currentTick && data.currentTick <
baseRanges[i].upperTick) {
data.currentRangeIndex = i;  // Overwrites on each match
data.hasCurrentRange = true;
}
unchecked { ++i; }
}
```

Subsequently, fixCurrentRangeAndRedistribute only fixes the range at currentRangeIndex and neglects redistributing for the previous positions that were also in the current range.

The impact is that the excess tokens from these other positions will remain idle in the MPM, until the owner withdraws them.

Currently, the protocol strategies enforce that the ranges list will not have any overlapping ranges. However, if an MPM owner decides to utilize custom strategies (with overlapping ranges), then they should be aware of this edge case.

Recommendation

Consider documenting this edge case for MPM owners who decide to utilize custom strategies.

Resolution

Gamma Team: Acknowledged.

I-09 | Single-Step Ownership Transfer Risk

Category	Severity	Location	Status
Best Practices	Info	GLOBAL	Acknowledged

Description

The contracts `MultiPositionManager`, `MultiPositionFactory`, `RelayerFactory`, `DynamicFeeHook`, `DynamicFeeLimitOrderHook`, `VolatilityDynamicFeeHook`, and `VolatilityDynamicFeeLimitOrderHook` rely on `OpenZeppelin` Ownable to guard sensitive actions, but ownership transfers take effect immediately.

If the owner key is compromised or a transfer is made by mistake, control moves instantly with no explicit acceptance step and no time for monitoring to react.

Recommendation

Consider implementing `OpenZeppelin` `Ownable2Step` in these contracts so ownership transfers require a separate `acceptOwnership` call before `onlyOwner` privileges move to the new owner.

Resolution

Gamma Team: Acknowledged.

I-10 | Unreachable Withdraw Flags Condition

Category	Severity	Location	Status
Superfluous Code	● Info	Relayer.sol: 440	Resolved

Description

In Relayer, `executeWithdrawal` contains a check that includes `(withdrawToken0Only && withdrawToken1Only)`.

However, `validateWithdrawalParams` function forbids setting both `withdrawToken0Only` and `withdrawToken1Only` at the same time and `executeWithdrawal` validates params before this branch.

This part of the condition is effectively unreachable under any allowed configuration.

Recommendation

Consider removing the unreachable branch.

Resolution

Gamma Team: Resolved.

I-11 | Limit Positions Minted Without Slippage Checks

Category	Severity	Location	Status
Warning	Info	PoolManagerUtils.sol: 98-112	Acknowledged

Description

Limit positions are minted using all remaining balances with `inMin = [0,0]`, meaning no slippage protection is applied.

This is not currently exploitable because limit positions are single-sided and use only leftover tokens, so no excess spending or token loss can occur.

If future changes introduce different execution assumptions, this could become a real slippage risk.

Recommendation

Ensure protocol team is aware of this, or consider documenting it.

Resolution

Gamma Team: Acknowledged.

I-12 | Compound Calls zeroBurn Twice Wasting Gas

Category	Severity	Location	Status
Gas Optimization	Info	DepositLogic.sol: 394	Acknowledged

Description

The compound() (and compoundSwap) function calls zeroBurnAllWithoutUnlock() twice:

```
function compound(uint256[2][] calldata inMin) external payable onlyOwnerOrRelayerOrFactory {
if (s.basePositionsLength > 0) {
poolManager.unlock(abi.encode(IMultiPositionManager.Action.ZERO_BURN, ""));
}
poolManager.unlock(abi.encode(IMultiPositionManager.Action.COMPOUND, abi.encode(inMin)));
}
```

Then, processCompound() calls it again:

```
function processCompound(...) external {
if (s.basePositionsLength == 0) return;
WithdrawLogic.zeroBurnAllWithoutUnlock(s, poolManager);
```

The second call collects zero fees since they were already collected. This wastes gas from looping through all positions twice and two separate UniswapV4 unlock calls.

Recommendation

Remove the zeroBurnAllWithoutUnlock call from processCompound().

Resolution

Gamma Team: Acknowledged.

I-13 | Burn Event Emitted When Shares Are Not Burned

Category	Severity	Location	Status
Events	Info	WithdrawLogic.sol: 190-191	Acknowledged

Description

In `WithdrawLogic.processWithdraw`, when `withdrawToWallet = false`, a Burn event is emitted even though shares are not actually burned. The main contract only burns shares when `withdrawToWallet = true`:

```
if (withdrawToWallet) {
_burn(owner(), shares);
}
```

This causes a mismatch between emitted events and actual state, misleading off-chain indexers and integrations that track share burns.

Recommendation

Either rename the event to accurately reflect the action (e.g., `LiquidityRemoved`), or only emit Burn when shares are actually burned.

Resolution

Gamma Team: Acknowledged.

I-14 | Withdraw Sends ETH Before Burn

Category	Severity	Location	Status
Reentrancy	Info	WithdrawLogic.sol: 154-160	Resolved

Description

In withdraw, ETH is transferred to the owner before shares are burned:
WithdrawLogic.processWithdraw():

```
s.currency0.transfer(to, amount0);  // ETH sent here
```

MultiPositionManager.withdraw():

```
if (withdrawToWallet) {
_burn(owner(), shares);  // Shares burned after
}
```

The owner could re enter deposit (which lacks nonReentrant) during the ETH callback, potentially receiving inflated shares since totalSupply hasn't been reduced yet.

Currently unexploitable because only the owner can deposit/withdraw, meaning they would be the only user affected. However, this violates CEI pattern and could become exploitable if the design changes to support multiple users.

Recommendation

Either add nonReentrant to deposit, or burn shares before transferring tokens.

Resolution

Gamma Team: Resolved.

I-15 | Stale Position Storage After Full Withdrawal

Category	Severity	Location	Status
Unexpected Behavior	Info	WithdrawLogic.sol: 130	Resolved

Description

When a full withdrawal occurs, the `WITHDRAW` action burns all liquidity but doesn't clear position storage (`basePositionsLength`, `basePositions`, `limitPositionsLength`, `limitPositions`).

As a result, the vault is left in a state where no liquidity exists, yet the position storage still reflects old ranges.

Subsequent operations (e.g. future deposit + rebalance) rely on these stale lengths for validation:

```
if (outMin.length != s.basePositionsLength + s.limitPositionsLength)
revert OutMinLengthMismatch();
```

Users must provide `outMin` arrays sized for stale positions with zero values, otherwise the transaction reverts. This also causes wasted gas due to unnecessary iteration over zero-liquidity positions.

Recommendation

Clear all position storage when `totalSupply` becomes zero.

Resolution

Gamma Team: Resolved.

I-16 | Missing Events For Configuration Updates

Category	Severity	Location	Status
Events	Info	GLOBAL	Acknowledged

Description

Several admin and configuration functions, as well as ETH flow functions, update important state without emitting events, reducing on-chain observability for monitoring, alerting, and incident response.

RelayerFactory.sol
• setAutomatedManagementFee updates automatedManagementFee with no event.

Relayer.sol
• setAutomatedManagementFee updates automatedManagementFee with no event,
• setWithdrawalParams updates state.withdrawalParams without emitting an event,
• setCompoundSwapParams updates state.compoundSwapParams without emitting an event,
• pause/unpause update state.isPaused without emitting Paused/Unpaused event declared in IRelayer.sol,
• fundContract/receive accept ETH with no event. withdrawFunds transfers ETH out with no event.
• executeRebalance/executeRebalanceSwap can use TWAP centering without emitting TwapCenterUsed (declared in IRelayer.sol).

MultiPositionFactory.sol
• setFeeRecipient updates feeRecipient with no event,
• setProtocolFee updates protocolFee with no event.

MultiPositionManager.sol
• claimFee triggers fee distribution without an event that records amounts/recipients.

Recommendation

Consider emitting dedicated events for these functions.

Resolution

Gamma Team: Acknowledged.

I-17 | MIN_BALANCE Check Counts Msg.value

Category	Severity	Location	Status
Unexpected Behavior	Info	Relayer.sol: 251	Resolved

Description

In `executeRebalanceSwap` and `executeCompoundSwap`, the initial funding check uses `address(this).balance`, which temporarily includes `msg.value`.

However, `msg.value` is later forwarded to MPM before `_reimburseGas`. If the relayer has little or no pre-funded ETH, the call can pass the `MIN_BALANCE` check using the caller-supplied `msg.value`, then revert during reimbursement due to insufficient remaining balance, reverting the entire transaction and causing wasted executions.

Recommendation

Check the relayer's balance excluding `msg.value` before proceeding.

Resolution

Gamma Team: Resolved.

I-18 | Missing Weight Validation In rebalanceSwap Path

Category	Severity	Location	Status
Unexpected Behavior	Info	RebalanceLogic.sol: 1490-1495	Resolved

Description

The rebalance() path validates that weights sum to 1e18:

```
if (ctx.weight0 + ctx.weight1 != 1e18) revert InvalidWeightSum();
```

However, _buildStrategyContext() used by the rebalanceSwap path lacks this validation. Malformed weights (e.g., weight0=0.8e18, weight1=0.8e18) can be passed into density calculations and be stored in lastStrategyParams.

Since only trusted callers can invoke this functions, impact is limited to user error.

Recommendation

Add the same validation in _buildStrategyContext() for consistency.

Resolution

Gamma Team: Resolved.

I-19 | Redundant Msg.value Check In deployRelayer

Category	Severity	Location	Status
Best Practices	Info	RelayerFactory.sol: 197-201	Resolved

Description

In `RelayerFactory.deployRelayer`, the function requires a minimum payment:
`if (msg.value < 0.001 ether) revert InsufficientPayment();`

Later, the function conditionally forwards ETH to the relayer:

```
if (msg.value > 0) {
(bool success,) = relayer.call{value: msg.value}("");
if (!success) revert InvalidAddress();
}
```

The `if (msg.value > 0)` check is redundant since the earlier validation guarantees `msg.value >= 0.001` ether. The condition will always be true.

Recommendation

Remove the redundant check:

```
(bool success,) = relayer.call{value: msg.value}("");
if (!success) revert InvalidAddress();
```

Resolution

Gamma Team: Resolved.

I-20 | BURN_AND_REBALANCE Enum Name Is Misleading

Category	Severity	Location	Status
Documentation	Info	WithdrawLogic.sol: 58C1-64C6	Resolved

Description

The `WithdrawPath` enum in `WithdrawLogic.sol` defines a path named `BURN_AND_REBALANCE`, but the implementation explicitly does not rebalance:

```
// Withdrawal path enum
enum WithdrawPath {
USE_CURRENT_BALANCE, // Step 1: sufficient idle balance
USE_BALANCE_PLUS_FEES, // Step 2: need zeroBurn for fees
BURN_AND_REBALANCE // Step 3: burn all + rebalance remaining
}
```

After burning, there is no rebalance, which the following comments confirm:

```
// NO REBALANCING - excess remains as unused balance
```

This creates confusion about the intended behavior and could mislead integrators.

Recommendation

Rename the enum to accurately reflect its behavior, e.g., `BURN_POSITIONS`

Resolution

Gamma Team: Resolved.

I-21 | Mint May Allow Overpaying At Manipulated Price

Category	Severity	Location	Status
Logical Error	● Info	PoolManagerUtils.sol: 176-178	Acknowledged

Description

The `_mintLiquidityForAmounts` function uses `minAmountIn` slippage protection instead of `maxAmountIn`.

This protects only against spending too little, but provides no protection against spending more than expected at an unfavourable price.

In most cases, the `outMin` slippage from burning will provide protection against this attack. However, burning existing positions (no `outMin` protection) can be skipped in the following cases:
1. The first deposit + rebalance on a pool with existing liquidity. Note that multiple MPMs (with different owners) can exist for a single pool.
2. Users deploying MPMs on popular trading pairs (WETH/USDC, etc.) with existing liquidity (i.e through direct `deployDepositAndRebalance` call).
3. The first deposit + rebalance after a full withdrawal (no existing positions to burn).

Combined with `slot0` spot price reads, attackers can sandwich this transaction:
1. Front-run: manipulate price with a swap.
2. Rebalance mints liquidity at the manipulated price.
3. Back-run: restore price and extract profit.

The slippage check still passes because `min tokens` were spent, even though they were spent at a worse exchange rate.

Unless `minIn` is always set high enough to prevent this attack, the transaction reverts easily due to small price changes, causing DoS.

Recommendation

Similar to Uniswap, consider also using `maxAmountIn` slippage when adding liquidity to a position.

Resolution

Gamma Team: Acknowledged.

I-22 | Slippage Bypass On Single-Token Rebalance

Category	Severity	Location	Status
MEV	Info	RelayerLogic.sol: 422-423	Acknowledged

Description PoC

In RelayerLogic, withdrawSingleToken function first calls MultiPositionManager.withdrawCustom (using the caller supplied outMin) to withdraw only token0 or token1 to the owner, then, if the other token still remains, calls rebalance to redeploy the leftover assets.

For this rebalance leg it constructs fresh outMin and inMin arrays and leaves them at their default zero values, effectively disabling slippage checks for both burning existing liquidity and minting new liquidity.

Because the rebalance flow derives allocations from the pool's current slot0 spot price, an attacker can sandwich the transaction by moving the price before the rebalance and restoring it after, extracting value from the vault and reducing the owner's remaining total value.

Since the rebalance mechanism in the withdrawSingleToken flow is currently unreachable due to M-26 issue, the severity of this finding has been reduced.

Recommendation

Accept and enforce outMin/inMin for the rebalance step and consider adding a TWAP deviation check.

Resolution

Gamma Team: Acknowledged.

I-23 | withdrawCustom Missing Shares Slippage

Category	Severity	Location	Status
Warning	Info	WithdrawLogic.sol: 253-282	Acknowledged

Description

The calculateSharesToBurn function in WithdrawLogic.sol uses the slot0 spot price to determine the amount of shares users must burn when calling withdrawCustom.

For PATH_1 (USE_CURRENT_BALANCE) and PATH_2 (USE_BALANCE_PLUS_FEES), there is no slippage protection on this calculation.

If the spot price moves unfavourably (natural volatility or attacker manipulation swapping token1 -> token0 and swap back at a small cost), the owner ends up burning significantly more shares than intended.

Since PATH 1 and PATH 2 withdraw from idle balances without burning positions, the outMin parameter provides no protection.

The user receives their requested tokens but permanently lose the excess shares burned.

Since the owner remains the sole shareholder, they can still withdraw all remaining vault value using their remaining shares. However, if the MPM ever supports multiple shareholders, this becomes problematic.

Recommendation

Ensure the protocol team is aware of this, in case there are any future changes. Consider adding a maxSharesToBurn parameter to withdrawCustom.

Resolution

Gamma Team: Acknowledged.

I-24 | Misleading StrategyParams Packing Comment

Category	Severity	Location	Status
Documentation	Info	SharedStructs.sol: 37-50	Resolved

Description

According to the comment, the `StrategyParams` struct is intended to fit in two storage slots, but the current field sizes exceed 32 bytes in the second slot.

The second slot contains two `uint120` fields (30 bytes) and three bool flags (3 bytes), totaling 33 bytes.

This forces the last flag to spill into a third slot. As a result, the existing comment claiming two-slot packing is misleading.

Recommendation

Change the mentioned comment or consider replacing the three bool fields with a single `uint8` flags value to keep the struct within two slots.

Resolution

Gamma Team: Resolved.

I-25 | Uint8 Loop Index Can Overflow

Category	Severity	Location	Status
Warning	Info	GLOBAL	Acknowledged

Description

Multiple loops that iterate over base positions use uint8 as the counter, which implicitly assumes `basePositionsLength <= 255`.

There is no explicit guard in the contract to enforce this bound, so if a strategy (especially a custom strategy) returns more than 255 ranges, the loop counter will overflow the loop will behave incorrectly.

Recommendation

Consider using `uint256` for loop counters or enforcing a hard upper bound on the number of ranges returned by strategies and revert if exceeded.

Resolution

Gamma Team: Acknowledged.

I-26 | Zero Liquidity Positions Skip Compounding

Category	Severity	Location	Status
Informational	Info	DepositLogic.sol: 197	Acknowledged

Description

Base positions with zero liquidity can be stored in the basePositions[] array. Unlike carpet positions which revert via InsufficientLiquidityForCarpet() when liquidity rounds to zero, base positions have no such validation.

Once stored, these positions are completely ignored — compound() distributes tokens proportionally based on existing token holdings:

```
if (positionToken0[i] != 0) {
amounts0[i] = FullMath.mulDiv(amount0ToDistribute, positionToken0[i],
totalToken0InPositions);
}
```

Positions with zero liquidity hold zero tokens, so they receive zero allocation and remain empty indefinitely. These dead positions consume gas during iteration in every operation (compound, withdraw, rebalance) while contributing nothing.

Recommendation

Consider allowing compound for these positions, or validate minimum liquidity/skip storing zero-liquidity positions during rebalancing.

Resolution

Gamma Team: Acknowledged.

I-27 | Lens Preview Uses totalSupply For outMin

Category	Severity	Location	Status
Informational	Info	RelayerLens.sol: 139	Resolved

Description

The lens computes withdrawal slippage bounds using `manager.totalSupply()`, but `Relayer.executeWithdrawal` withdraws only the MPM owner's shares. Since shares are `ERC20` and deposit can mint to arbitrary to addresses, the owner may not hold the full supply.

This inconsistency can overstate the shares actually withdrawn, producing misleading `outMin` values and inaccurate withdrawal previews.

Recommendation

Update the preview to mirror the actual withdrawal logic by using the MPM owner's share balance (`balanceOf(Ownable(manager).owner())`) when calculating `outMinForShares`.

Resolution

Gamma Team: Resolved.

I-28 | Native ETH Transfers Use Transfer

Category	Severity	Location	Status
Best Practices	Info	MultiPositionManager.sol	Resolved

Description

Native ETH payouts use transfer, which forwards only 2300 gas and can fail for valid contract recipients. This creates a dos risk: fee claims or withdrawals can revert and leave ETH stranded.

Recommendation

Use a safe call pattern for native transfers (`(bool ok, ) = recipient.call{value: amount}("")`) and require success.

Resolution

Gamma Team: Resolved.

I-29 | Asymmetric Limit Trigger At Lower Boundary

Category	Severity	Location	Status
Suggestion	Info	RelayerLogic.sol: 235-244	Acknowledged

Description

`_checkLimitTickTrigger` decides whether the current price is inside the active limit range using only `slot0.tick`. Uniswap v4 documents an edge case where `slot0.tick` can be one less than the tick implied by `sqrtPriceX96` when the price is exactly on a lower tick boundary.

In that state, the observed `currentTick` equals `lowerTick - 1` even though the price is on the lower boundary.

The implementation uses asymmetric inequalities for below/above distance checks. Using `>` below the range but `>=` above makes the threshold asymmetric by one tick.

It requires `lowerTick - currentTick` to exceed `limitDeltaTicks` to trigger below, but triggers as soon as `currentTick - upperTick` reaches `limitDeltaTicks` above.

This subtle one-tick directional buffer may surprise integrators expecting symmetric distance checks.

Recommendation

Consider explicitly treating the lower-boundary edge case as inside. When `currentTick == lowerTick - 1` and `sqrtPriceX96 == TickMath.getSqrtPriceAtTick(lowerTick)`, consider treating the price as on-boundary and do not trigger.

Resolution

Gamma Team: Acknowledged.

Remediation Findings & Resolutions V1

ID	Title	Category	Severity	Status
M-01	Inaccurate OP Stack L1 Fee Refund	Unexpected Behavior	Medium	Resolved
M-02	Carpet Mode Disable Mint Slippage	Frontrunning	Medium	Resolved
M-03	Center Tick Rounding Causes Strategy Overflow	Logical Error	Medium	Resolved
L-01	Rebalance Liquidity Overflow At Edge States	DoS	Low	Resolved
L-02	withdrawCustom Burns Shares For Zero Output	Logical Error	Low	Resolved
L-03	Paused Relayer Still Charges Increased Fee	Logical Error	Low	Resolved
L-04	Double-Counted Amounts In Withdraw Burn Event	Events	Low	Resolved
I-01	Dead Fallback Branches In withdrawCustom Flow	Informational	Info	Acknowledged
I-02	Uncapped Gas Reimbursement	Suggestion	Info	Acknowledged
I-03	withdrawCustom Burns Shares For Claimable Fee	Logical Error	Info	Acknowledged
I-04	Unnecessary S.fee != 0 Guard In processClaimFee	Superfluous Code	Info	Resolved
I-05	Fee Change Retroactively Affects Unclaimed Fees	Suggestion	Info	Acknowledged

M-01 | Inaccurate OP Stack L1 Fee Refund

Category	Severity	Location	Status
Unexpected Behavior	Medium	Relayer.sol: 678-684	Resolved

Description

The relayer tries to reimburse OP Stack full transaction cost by adding an L1 data fee to the normal L2 gas refund, by calling the OP Stack `GasPriceOracle` with the current call data.

However, this is inaccurate for two reasons:

• `getL1Fee` is designed to estimate the L1 data cost from the bytes of an unsigned, RLP-encoded transaction, but the relayer passes only `msg.data`. Because this omits the transaction envelope bytes (transaction type prefix, nonce, gas limit, `maxFeePerGas`/`maxPriorityFeePerGas`, to, value, access list, etc.), the oracle is not pricing the same data that will actually be published to L1, so the L1 fee estimate will be consistently too low.

• `gasUsed` is computed before calling the oracle, so the L2 gas spent to compute `l1Fee` is not included in `gasUsed` and is never reimbursed.

Consequently, automation services can be systematically under-reimbursed, causing rebalances and withdrawals to stop running reliably.

Recommendation

Reorder the accounting so the oracle call is included in `gasUsed`, and estimate L1 fee using a more accurate input, commonly by appending a small constant padding to `msg.data` before calling `getL1Fee`.

Resolution

Gamma Team: Resolved.

M-02 | Carpet Mode Disable Mint Slippage

Category	Severity	Location	Status
Frontrunning	Medium	RebalanceLogic.sol: 1374	Resolved

Description

During `processRebalanceInCallback`, the contract validates the length of the user-supplied `inMin` array. When `rebalanceParams.useCarpet` is true and `inMin.length != baseRanges.length`, the code discards the caller's `inMin` and replaces it with a zero-filled array.

Since `inMin` is mint-side slippage protection, filling it with zeros effectively disables the caller's intended safeguards. A rebalance can then proceed under worse than expected execution conditions without reverting.

Consequently, a MEV actor can sandwich the rebalance and it may still succeed because mint `inMin` slippage checks have been bypassed.

Recommendation

Do not auto-zero slippage parameters on length mismatch. Consider always reverting when `inMin.length != baseRanges.length`, and only accept `inMin.length == 0` as an explicit no slippage protection signal.

Resolution

Gamma Team: Resolved.

M-03 | Center Tick Rounding Causes Strategy Overflow

Category	Severity	Location	Status
Logical Error	Medium	RebalanceLogic.sol: 153-160	Resolved

Description

The protocol uses a sentinel value `type(int24).max` to derive the strategy center from the pool's current tick when executing rebalances. The relayer constructs `RebalanceParams` with this value in `constructRebalanceParams` function.

Downstream, the `rebalance` and `rebalanceSwap` flows interpret this sentinel by reading `currentTick` from slot0 and rounding it down to a `tickSpacing` multiple.

This rounding is not constrained to Uniswap's usable tick range (multiples of `tickSpacing` within [MIN_TICK, MAX_TICK]). When `currentTick` is close to the minimum tick boundary, the floor rounding can produce a `ctx.center` below `minUsableTick(tickSpacing)`.

That invalid `ctx.center` is then passed to `strategy.generateRanges`, where range generation can compute an invalid span (right bound below left bound).

When several strategy converts this signed span into a `uint256`, the negative value becomes a huge number and later arithmetic overflows, causing the rebalance to revert.

Recommendation

Clamp the derived `ctx.center` to Uniswap's usable tick range before calling the strategy.

Resolution

Gamma Team: Resolved.

L-01 | Rebalance Liquidity Overflow At Edge States

Category	Severity	Location	Status
DoS	Low	RebalanceLogic.sol: 1381	Resolved

Description PoC

The rebalance flow can revert while minting liquidity under specific edge-state combinations. In the rebalance callback path, liquidity is computed using `LiquidityAmounts.getLiquidityForAmounts()` and must fit into `uint128`.

In edge states (price near usable tick boundaries, edge-aligned/narrow limit ranges, and large effective per-range allocations), computed liquidity can exceed `type(uint128).max`, causing a revert (liquidity overflow).

When this happens, the rebalance transaction reverts, liquidity is not redeployed, and repeated attempts can continue to fail under similar conditions, creating a temporary DoS.

Recommendation

Add a pre-mint bound check before calling `getLiquidityForAmounts()` and cap or rescale per-range allocations whenever projected liquidity would exceed `type(uint128).max`.

Resolution

Gamma Team: Resolved.

L-02 | withdrawCustom Burns Shares For Zero Output

Category	Severity	Location	Status
Logical Error	Low	PoolManagerUtils.sol: 299	Resolved

Description

In WithdrawLogic, processWithdrawCustom function computes sharesBurned for the requested amounts and then, when idle balances and fees are insufficient, takes the PATH 3 branch. In this branch it burns a pro-rata portion of positions and then transfers the idle balances.

The underlying burn is performed by computing how much liquidity corresponds to the burned shares. However, in PoolManagerUtils.burnLiquidityForShare, the amount of liquidity to burn is truncated with integer division, so no liquidity is burned and no tokens are released.

```
uint256 liquidityForShares = FullMath.mulDiv(liquidity, shares, totalSupply);
```

The withdrawCustom then computes outputs from the post-burn idle balances, which can remain 0, and the caller still burns sharesBurned after processWithdrawCustom returns.

The protocol intends each MPM to have a single trusted shareholder, which reduces the impact of this finding - however this assumption is not enforced at the contract level because shares are standard ERC20 and deposits can mint to arbitrary to address.

Recommendation

Prevent share burns when the withdrawal produces no assets by reverting PATH 3 when both outputs are zero.

Resolution

Gamma Team: Resolved.

L-03 | Paused Relayer Still Charges Increased Fee

Category	Severity	Location	Status
Logical Error	Low	Relayer.sol: 572	Resolved

Description

When `RelayerFactory.deployRelayer` is called, it sets `automatedManagementFee` on the MPM via `setFee`. This increases the protocol fees from 5% to 10%.

If the MPM owner later pauses the relayer, automated rebalances stop but the increased fee remains.

Fees accumulated during the paused period are still split at the higher `automatedManagementFee` rate when claimed, even though no automation service is being provided.

Recommendation

Consider rebooting the fee to the factory's default `protocolFee` when the relayer is paused (ensure to collect fees just before changing the protocol fee, so previous accumulated fees still apply), and increase it again when unpaused.

Resolution

Gamma Team: Resolved.

L-04 | Double-Counted Amounts In Withdraw Burn Event

Category	Severity	Location	Status
Events	Low	WithdrawLogic.sol: 178-191	Resolved

Description

In processWithdraw, the withdrawToWallet = false path calculates unusedAmounts as a proportional share of balanceOfSelf(). However, at this point balanceOfSelf() already includes the tokens received from the burn callback (settled via _closePair).

Adding these to the already-set amount0/amount1 double counts the burned amounts in the emitted Burn event. The withdrawToWallet = true path avoids this by transferring burned amounts before calculating unused balances.

Recommendation

Subtract the burned amounts from balanceOfSelf() before calculating unused amounts, or compute unused amounts only from the pre-existing idle balance.

Resolution

Gamma Team: Resolved.

I-01 | Dead Fallback Branches In withdrawCustom Flow

Category	Severity	Location	Status
Informational	● Info	WithdrawLogic.sol: 402-409	Acknowledged

Description

In `WithdrawLogic`, `calculateSharesToBurn` includes a fallback for `poolValueInToken1 == 0` intended to handle non zero `token1` values.

However, in the real `withdrawCustom` flow these branches are effectively unreachable, as `processWithdrawCustom` enforces `amount1Desired <= total1` before calling `calculateSharesToBurn`.

```
if (params.amount1Desired > pathInfo.total1) revert InsufficientBalance();
```

`processWithdrawCustom` passes `pathInfo.total1` into `calculateSharesToBurn` as `pool1`. When the fallback triggers (`poolValueInToken1 == 0`), `pool1` must be 0, which forces `amount1Desired` to also be 0 due to the precondition above.

Therefore, the fallback logic intended to handle nonzero `token1` (the price conversion branches gated by `amount1Desired != 0` or `pool1 != 0`) cannot be executed in the `withdrawCustom` path.

Recommendation

Move the nonzero-`token1` fallback logic to preview-only code or document that it cannot occur in `withdrawCustom` flow.

Resolution

Gamma Team: Acknowledged.

I-02 | Uncapped Gas Reimbursement

Category	Severity	Location	Status
Suggestion	Info	Relayer.sol: 640	Acknowledged

Description

The relayer reimburses automation callers with no cap on the effective gas price or per-call reimbursement.

Because execution is restricted to `AUTOMATION_SERVICE_ROLE`, this is a privileged-abuse path rather than a permissionless exploit; however, a malicious or compromised automation address can still overpay gas and extract disproportionate ETH reimbursements, draining the relayer balance and causing automation to stop once balance falls below `minBalance`.

Recommendation

Consider adding configurable caps on reimbursable gas price and per-call reimbursement to limit overpayment risk.

Resolution

Gamma Team: Acknowledged.

I-03 | withdrawCustom Burns Shares For Claimable Fee

Category	Severity	Location	Status
Logical Error	Info	WithdrawLogic.sol: 260-261	Acknowledged

Description

calculateSharesToBurn computes shares to burn using fee-inclusive totals from getTotalAmounts.

When the owner uses withdrawCustom and the withdrawal is satisfied partly by claimable fees (via USE_BALANCE_PLUS_FEES or BURN_AND_WITHDRAW paths), shares are burned proportional to the full withdrawal amount including the fee portion.

Since the owner can claim fees via claimFee() without burning any shares, this effectively costs the owner shares for value they are entitled to for free.

Since the owner is the sole shareholder, they're not losing value to anyone, just burning more shares than needed.

Recommendation

Exclude the owner's claimable fee portion from the share calculation, or automatically call claimFee before computing sharesBurned so that fee value is already extracted before the withdrawal share math is applied.

Resolution

Gamma Team: Acknowledged.

I-04 | Unnecessary S.fee != 0 Guard In processClaimFee

Category	Severity	Location	Status
Superfluous Code	Info	WithdrawLogic.sol: 595	Resolved

Description

In `processClaimFee`, `if (s.fee != 0)` is checked prior to executing owner transfer logic. However, `s.fee` is guaranteed non-zero (explicitly requires newFee != 0 in setFee) since `totalFee0 / s.fee` would revert due to division-by-zero.

Recommendation

Remove the redundant check.

Resolution

Gamma Team: Resolved.

I-05 | Fee Change Retroactively Affects Unclaimed Fees

Category	Severity	Location	Status
Suggestion	Info	MultiPositionManager.sol: 376	Acknowledged

Description

`setFee` updates `s.fee` immediately, but fee splits are calculated at claim time using the current `s.fee` as a denominator.

If fees accumulated under a previous rate are not claimed before changing the fee, the new rate applies retroactively.

For example, fees accumulated at `s.fee = 20` (5% treasury) would be split at 10% if `s.fee` is changed to `10` before claiming (i.e., during relayer deployment).

Recommendation

Call `claimFee` before updating the fee, either by enforcing it within `setFee` itself or documenting it as a required precondition. In `RelayerFactory::deployRelayer`, trigger a fee claim on the MPM before calling `setFee`.

Resolution

Gamma Team: Acknowledged.

Remediation Findings & Resolutions V2

ID	Title	Category	Severity	Status
L-01	TickLiquidityOverflow In Rebalance	DoS	Low	Acknowledged
I-01	Donation Griefing Via Tight inMin	Warning	Info	Acknowledged

L-01 | TickLiquidityOverflow In Rebalance

Category	Severity	Location	Status
DoS	Low	LiquidityAmountsCapped.sol	Acknowledged

Description PoC

The rebalance flow can still revert during mint under edge-state combinations due to per-tick liquidity capping, causing `TickLiquidityOverflow` in Uniswap v4.

In the rebalance callback path (`PoolManagerUtils._mintLiquidityForAmounts`), liquidity is derived via `LiquidityAmountsCapped.getLiquidityForAmountsCapped()` and capped to `int128.max`, but not capped against the pool's `maxLiquidityPerTick` constraint.

When price is near extreme usable ticks and ranges become narrow/edge-aligned (for example, near 887271 -> 887272), the computed `liquidityDelta` can remain valid for `uint128` yet still exceed per-tick headroom, so `poolManager.modifyLiquidity(...)` reverts with `TickLiquidityOverflow`.

Recommendation

Cap rebalance mint liquidity by tick headroom, not just `uint128` bounds.

Before `modifyLiquidity`, compute `maxLiquidityPerTick` from `tickSpacing` and clamp `liquidityDelta` to the smaller remaining headroom across `tickLower` and `tickUpper`. If headroom is zero, skip that mint.

Resolution

Gamma Team: Acknowledged.

I-01 | Donation Griefing Via Tight inMin

Category	Severity	Location	Status
Warning	● Info	PoolManagerUtils.sol: 208-209	Acknowledged

Description

The `MultiPositionManager` mints new liquidity using the full on-contract balances of the pool currencies (including any unsolicited `ERC20` or ETH sent to the contract).

If an operator or automation service supplies tight `inMin` values for rebalance minting, a third party can grief those rebalances by donating a small amount of one of the pool tokens immediately before execution, shifting the mint outcomes and a base position to fall below the precomputed `inMin` thresholds.

The minting path enforces per-position minimums and will revert the entire rebalance when any position under-mints either token.

Consequently, rebalances can be DoS'd as long as the attacker is willing to donate assets to the vault, forcing repeated reverts until `inMin` is loosened.

Recommendation

Be aware of this vector. While the attack is infeasible since an attacker would burn funds, a tightly set `inMin` could still be violated by a donation and cause a `SlippageExceeded` revert.

Resolution

Gamma Team: Acknowledged.

Disclaimer

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian’s position is that each company and individual are responsible for their own due diligence and continuous security. Guardian’s goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract’s safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.

To learn more, visit https://guardianaudits.com

To view our audit portfolio, visit https://github.com/guardianaudits

To book an audit, message https://t.me/guardianaudits